"Multi-Source Knowledge API" - Take-Home Assignment

> **Time Estimate:** 4-6 hours
>
> **Deadline:** 3 days

CORE Core Requirements (Must Have)

Build a conversational RAG API that supports multiple document formats:

Backend:

- Ingest documents: PDF, DOCX, CSV, JSON
- Store chunks with embeddings in vector DB
- Session management: track conversations with `session_id`
- Store chat history in database (user query + AI response + sources)

API Endpoints:

- `POST /ingest` - upload and process documents (async with job status)
- `POST /chat` - ask questions with session context
- `GET /sessions/{id}` - retrieve conversation history

AI Engineering:

- Implement a chunking strategy (explain your approach in README)
- Vector-based semantic search
- Return answers with source citations

Requirements:

- Database schema for sessions, documents, chunks, chat history
- Basic error handling and validation
- Docker setup
- README with setup instructions + architectural decisions

TIER 2 Intermediate (Pick 1-2)

- **Hybrid Search:** Combine vector similarity + keyword search (BM25), implement fusion scoring
- **Query Enhancement:** Rewrite/expand queries before retrieval using LLM
- **Smart Routing:** Different retrieval strategies for structured (CSV/JSON) vs unstructured (PDF/DOCX) data
- **Follow-up Context:** Handle "what about X?" queries using conversation history

- **Streaming Responses:** Stream chat responses with SSE

TIER 3 Advanced (Optional, Pick 1)

- **Evaluation Suite:** Include test queries with ground truth, measure precision/recall
- **Re-ranking:** Post-retrieval re-ranking using cross-encoder or LLM
- **Caching Layer:** Cache embeddings and search results with Redis

Evaluation Criteria

✅ Core tier complete and working (baseline)

⭐ Code quality: Clean architecture, tests, documentation

⭐⭐ 1-2 Tier 2 features implemented well

⭐⭐⭐ Advanced features + thoughtful trade-offs explained

We care more about depth in a few areas than surface-level implementation of everything.

Required Documentation (in README)

Architecture Overview

- System design diagram (can be simple ASCII or image)
- Component responsibilities

Technical Decisions & Trade-offs

- **Database choice:** Why did you pick X over Y?
- **Vector DB:** Why pgvector/Chroma/FAISS? Trade-offs?
- **Chunking strategy:** Size, overlap, reasoning
- **Embedding model:** Which one and why?
- **Design patterns:** What patterns did you use and why? (e.g., Repository, Factory, Strategy)
- **Async processing:** How did you handle ingestion jobs?

What You'd Do Differently in Production

- Scalability concerns
- Security improvements
- Monitoring/observability
- Cost optimizations

Additional Scope & Communication

> - You may use GitHub to develop the project and share it with our team after completion.
> - Please add the following GitHub users as collaborators: **huseyinevecen** and **yigitsimsek**.

- For any questions or clarifications, contact:
  - huseyin.evecen@sagalegal.io
  - yigit.simsek@sagalegal.io
- It is preferable to use **JavaScript and Node.js** to build the API.

Submission Guidelines

1. Create a GitHub repository with your code
2. Add **huseyinevecen** and **yigitsimsek** as collaborators
3. Include a comprehensive README.md with:
   - Setup instructions
   - API documentation
   - Architecture overview
   - Design decisions and trade-offs
4. Provide a `.env.example` file with required environment variables
5. Email the repository link to **huseyin.evecen@sagalegal.io** and **yigit.simsek@sagalegal.io**

Questions?

If you have any questions about the assignment, feel free to reach out. We're happy to clarify requirements or provide guidance on technical decisions.

Contact: huseyin.evecen@sagalegal.io and yigit.simsek@sagalegal.io

---

SAGA | Enabling your AI journey